

Cargoservice — Sprint1

ISS 2026 — VALERIA CAPACCI & GINEVRA PENTOLI

[GOAL](#) [PROBLEM ANALYSIS](#) [TEST PLANS](#) [PROJECT](#) [DEPLOYMENT](#) [SPRINT2](#)

Goal

Realizzare il sottosistema che gestisce l'accettazione/rifiuto di una richiesta di carico, il timeout dei 30 secondi, e l'interazione reale con lo `IOPort` : pulsante, sensore, display e LED. `cargoservice` riceve `loadrequest` , interroga `Hold` , risponde con `loadaccepted` / `loadrejected` e transita da Idle a Engaged; se il timeout scade torna a Idle liberando lo slot.

Problem analysis

Scelta della tecnologia di implementazione dell'IOPort

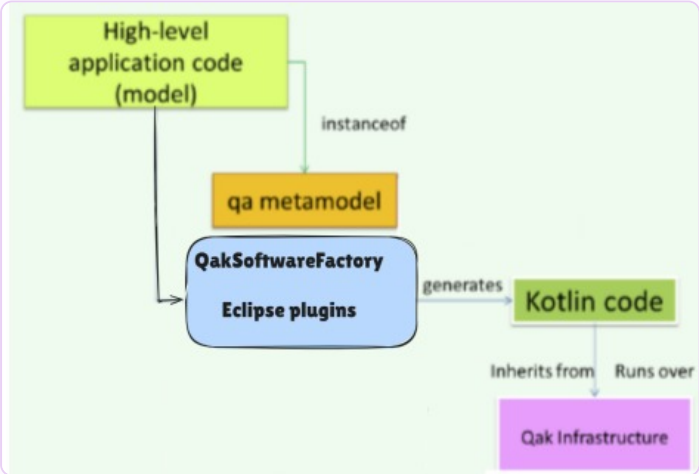
L'IOPort è stato individuato, fin dalla fase di analisi dei requisiti iniziale, come nodo architetturale a sé stante rispetto a `cargoservice` : comunica con l'utente finale, ma la sua implementazione non deve necessariamente ricalcare lo stack tecnologico del resto del sistema. Le due alternative concretamente valutate sono state un dispositivo dedicato con touchscreen fisico (pannello embedded, con firmware proprio) oppure una WebGUI fruibile da browser.

Dopo averne discusso con il committente, questo ha indicato una preferenza esplicita per la WebGUI. La scelta è comunque coerente con l'analisi svolta: a fronte di una minore possibilità di intervenire direttamente sull'hardware, la WebGUI riduce i costi di manutenzione, evita di introdurre un secondo stack tecnologico nel sistema e permette che lo stato della hold e delle richieste (come richiesto) sia visualizzabile da più utenti contemporaneamente, senza vincolare la soluzione a un singolo dispositivo fisico installato all'IOPort.

TCP come protocollo di trasporto interno di qak e scelta del canale WebSocket per la GUI

Qak adotta TCP come protocollo di trasporto per la comunicazione tra attori su `Context` distinti: ogni `Context` apre un socket TCP in ascolto sulla porta dichiarata (`host=localhost port=8050` per `ctxcargoservice` , vedi Deployment), e la infrastruttura di runtime serializza gli `IAppMessage` scambiati tra attori in un formato proprio, li trasmette sul socket e li deserializza a destinazione, ricostruendo l'invocazione sull'attore ricevente. Questo protocollo di serializzazione/framing è interno al runtime e opera esclusivamente al livello di astrazione offerto dal modello ad attori, tramite primitive come `sendMsgToMyself` : lo sviluppatore non vede mai il socket sottostante.

Proprio perché il TCP di qak è mascherato dietro questa astrazione, non è un canale riutilizzabile da un client qualsiasi: è un protocollo applicativo pensato per far dialogare tra loro istanze del runtime, con un formato di serializzazione noto solo alle due estremità qak-qak. Un browser non può costruire né interpretare un `IAppMessage` in questo formato.



Il canale verso il browser va quindi progettato a parte, e deve soddisfare un requisito bidirezionale: inviare comandi verso il sistema e ricevere aggiornamenti di stato in tempo reale (stato, hold, messaggio). Sono state valutate, e di conseguenza scartate, le seguenti alternative di trasporto disponibili per una comunicazione lato browser:

Alternativa	Limite rispetto al caso d'uso
TCP puro	fornisce solo il trasporto a stream di byte, senza semantica di messaggio: andrebbe reimplementato da zero il framing dei messaggi applicativi
CoAP	pensato per dispositivi IoT vincolati con un modello prevalentemente request/response leggero; non adatto a una comunicazione bidirezionale continua con un browser
MQTT	supporta publish/subscribe bidirezionale, ma richiede un broker come componente architetturale aggiuntivo e non è nativamente supportato dai browser senza un bridge (es. MQTT over WebSocket)

Il protocollo WebSocket, che stabilisce un canale full-duplex su una singola connessione TCP, è nativamente supportato da qualunque browser senza componenti intermedi, e risulta quindi funzionalmente adeguato a un caso d'uso di tipo dashboard/notifica live, come in questo caso, in quanto stato della hold e delle richieste deve essere da requisito visualizzabile da più utenti contemporaneamente. Per questo motivo `IOPortWsAdapter` espone un endpoint `/ws/ioport` tramite un server Javalin dedicato (vedi [IOPortWsAdapter.java](#)), con broadcast verso l'insieme dei client connessi.

Instradamento dei messaggi

Nel protocollo WebSocket un messaggio è una sequenza di byte o testo priva di header applicativi: WebSocket fornisce il canale, non la semantica di ciò che vi transita. Poiché la GUI deve poter inviare più tipi di comando sulla stessa connessione, serve che le due parti si accordino su un formato condiviso in cui codificare tipo di messaggio e payload.

A livello di implementazione tale formato è stato realizzato in JSON, con un envelope che porta un campo discriminante `type`. La scelta di JSON rispetto ad alternative è motivata dalla leggerezza, dalla natura testuale nativamente compatibile con WebSocket, e dalla facilità di parsing sia lato client (`JSON.parse` in JavaScript, senza librerie aggiuntive) sia lato server (`JSONParser` già disponibile).

L'envelope JSON viene decodificato da `toDispatch()` per determinare il messaggio qak da generare. Questa soluzione, standard per il routing su protocolli privi di semantica applicativa, riduce la complessità sia lato client sia lato server mantenendo un'unica connessione persistente per l'intero dispositivo simulato.

IOPortWsAdapter.java

```
static String[] toDispatch(String raw) throws Exception {
    JSONObject json = (JSONObject) new JSONParser().parse(raw);
    String type = (String) json.get("type");
    if ("pushButton".equals(type)) {
        return new String[] { "pushButton", "pushButton(0)" };
    } else if ("setOccupied".equals(type)) {
        boolean value = Boolean.TRUE.equals(json.get("value"));
        return new String[] { "setOccupied", "setOccupied(" + value + ")" };
    }
    throw new IllegalArgumentException("unknown message type: " + type);
}
```

Iniezione del messaggio nell'attore

`IOPortWsAdapter` opera sul thread Jetty associato alla connessione WebSocket, distinto dal thread di esecuzione dell'attore. L'invocazione diretta di metodi che modificano lo stato interno dell'attore da tale thread violerebbe il principio di non condivisione della memoria su cui si fonda il modello ad attori di qak. La soluzione adottata prevede che l'adapter, ricevuto un riferimento all'attore alla propria creazione (`IOPortWsAdapter(this, 7070)`), costruisca un `IApplMessage` e lo instradi tramite `sendMsgToMyself` nella coda dell'attore stesso.

[sprint1.qak](#)

```
QActor ioport context ctxcargoservice
  withobj status using "utils.cargoservice.IOPortStatus()" {

  [# var WsAdapter = utils.cargoservice.IOPortWsAdapter(this, 7070) #]
```

[IOPortWsAdapter.java](#)

```
private void sendToActor(String msgId, String content) {
    IAppIMessage dispatch = CommUtils.buildDispatch(SENDER, msgId, content,
IOPORT_NAME);
    ioport.sendMsgToMyself(dispatch);
}
```

Context unico per Sprint1

`cargoservice` e `ioport` sono stati mantenuti all'interno dello stesso `Context ctxcargoservice`, scelta di semplificazione esplicitamente temporanea: la separazione dei Context per nodo fisico è demandata allo Sprint2.

Test plans

Per lo Sprint1 ci concentriamo sul verificare che, a fronte di un input noto (un messaggio inviato all'attore), il sistema produca l'output atteso (una reply o un aggiornamento del display), indipendentemente da come è implementato internamente. Ogni caso di test corrisponde quindi a una sequenza osservabile di messaggi qak, verificabile leggendo la reply ricevuta da `cargoservice` o il payload broadcast da `IOPortWsAdapter` verso i client WebSocket.

Caso	Precondizione	Azione	Risultato atteso
Richiesta accettata	almeno uno slot <code>FREE</code> in <code>hold</code> ; sensore non occupato	<code>pushButton</code> su <code>ioport</code>	<code>cargoservice</code> risponde <code>loadaccepted(SLOT,HOLD)</code> con uno <code>SLOT</code> valido; transita da <code>idle</code> a <code>engaged</code> ; il display mostra "Richiesta accettata – slot X"
Rifiuto: retrylater	sensore forzato a occupato (<code>setOccupied(true)</code>)	<code>pushButton</code>	<code>loadrejected(retrylater,HOLD)</code> ; il display mostra "Richiesta rifiutata: retrylater"
Rifiuto: full	tutti e 4 gli slot già <code>RESERVED</code> / <code>OCCUPIED</code> (4 richieste precedenti accettate)	<code>pushButton</code> con sensore libero	<code>loadrejected(full,HOLD)</code>
Timeout dei 30 s	richiesta appena accettata, <code>cargoservice</code> nello stato <code>engaged</code>	nessuna interazione per 30000 ms (<code>Transition t1 whenTime</code>)	<code>cargoservice</code> rilascia lo slot (<code>hold.releaseReserved()</code>), torna <code>idle</code> e invia a <code>ioport</code> un <code>updateDisplay</code> con stato <code>IDLE</code> e messaggio "Timeout scaduto: slot liberato, torno idle"

Trattandosi di un sistema che espone la propria interazione tramite un pannello GUI, i casi sono al momento eseguibili manualmente dal pannello di controllo, osservando le reply e i log colorati prodotti dagli attori.

Project

Il sistema `sprint1` (vedi [sprint1.qak](#)) definisce un unico `Context ctxcargoservice` (`host=localhost port=8050`) che ospita entrambi gli attori.

L'attore `cargoservice` incapsula la logica di gestione degli slot tramite `withobj hold using "utils.cargoservice.Hold()"` (vedi `Hold.java`). La classe `Hold` mantiene uno stato indipendente dal modello ad attori ed espone un'interfaccia a metodi che costituisce l'unico punto di accesso allo stato interno.

L'attore `ioport` incapsula lo stato del sensore tramite `withobj status using "utils.cargoservice.IOPortStatus()"` (vedi `IOPortStatus.java`), limitato a un singolo campo booleano `occupied`, dimensionato sulla singola informazione effettivamente prodotta dal sensore. Nello stato iniziale dell'attore viene istanziato l'adattatore WebSocket (`[# var WsAdapter = utils.cargoservice.IOPortWsAdapter(this, 7070) #]`), che costituisce l'unico punto di interfacciamento tra il dominio qak e il client browser: ogni transizione di `ioport` che richiede un aggiornamento della GUI invoca `WsAdapter.updateDisplay(...)` o `WsAdapter.notifySensor(...)`, entrambi risolti internamente in un broadcast JSON verso i client connessi (vedi `IOPortWsAdapter.java`).

Il livello di presentazione (`ioport_gui.html`, `wscontrol.js`) non contiene logica di business. Il countdown di 30 secondi visualizzato lato client è puramente rappresentativo (si avvia alla pressione del pulsante e disabilita il controllo per la durata corrispondente) e non costituisce la fonte di verità del timeout, gestito interamente lato server dalla `Transition t1 whenTime 30000 -> disengaged` di `cargoservice`. Questa separazione evita che un client disconnesso o con latenza elevata possa disallineare lo stato effettivo del sistema rispetto a quanto visualizzato.

Deployment

Il progetto si avvia dalla cartella `sprint1` tramite `gradlew run`

Il sistema espone due porte distinte, con responsabilità separate:

Porta	Componente	Funzione
8050	<code>Context ctxcargoservice</code>	porta di infrastruttura del Context qak, riservata all'eventuale comunicazione remota tra attori; non utilizzata nello Sprint1 poiché <code>cargoservice</code> e <code>ioport</code> condividono lo stesso Context
7070	Server Javalin embedded (<code>IOPortWsAdapter</code> , istanziato dall'attore <code>ioport</code>)	traffico HTTP/WS della GUI: redirect da <code>/</code> a <code>/ioport_gui.html</code> , endpoint WebSocket su <code>/ws/ioport</code>

Il pannello di controllo è raggiungibile all'indirizzo `http://localhost:7070/ioport_gui.html` a sistema avviato.



Definizione dello Sprint2

Lo Sprint1 ha chiuso il core business (accettazione/rifiuto, timeout, IOPort come attore reale collegato a una GUI). Lo Sprint2 dovrà estendere il sistema con i componenti rimasti fuori da Sprint1:

REQUISITI

- Introdurre l'attore `sonar` : emissione degli eventi `sonarcontainer` / `sonarfault` a partire dalle letture di distanza, e gestione dello stato *Out of service* ($D > D_{FREE}$ per almeno 3 s) sul display dello IOPort.
- Progettare e realizzare l'attore `cargorobot` , che orchestra il movimento del DDR appoggiandosi al servizio esterno `RobotSmart26` : raggiungere l'IOPort, prelevare il container, passare da `slot5` (marcatura) allo slot riservato, tornare in HOME.
- Modellare il `marker` : segnalazione (interna al nodo `cargoservice`) del completamento della marcatura in `slot5` , prima che il robot possa prelevare il container.
- Separare i Context per nodo fisico (`cargoservice` , `cargorobot` , `sonar` , `ioport`), oggi ancora unificati in `ctxcargoservice` per semplicità.



Valeria Capacci
valeria.capacci@studio.unibo.it



Ginevra Pentoli
ginevra.pentoli@studio.unibo.it

GIT: github.com/valecapa/TemaFinaleISS2026